

STRUTTURE DATI ELEMENTARI

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



- **Struttura dati:**
 - Definisce come i dati sono **logicamente organizzati**
 - Definisce le **operazioni** per accedere e modificare i dati
- Descrive **come** i dati sono organizzati non **quali** dati sono memorizzati
 - Esempio: la struttura lista può contenere interi oppure stringhe
- Vedremo tre tipologie di strutture dati elementari
 - Liste concatenate (Linked List)
 - Pile (Stack)
 - Code (Queue)

PRELIMINARI: PROTOTIPO VS IMPLEMENTAZIONE

■ Prototipo

- Descrive possibili valori ed operazioni di una struttura dati
- Nasconde i dettagli implementativi
- Permette al programmatore di implementare la struttura dati
- Permette all'utente di capire come usare la struttura dati

■ Implementazione

- Realizzazione di una struttura dati con qualche linguaggio di programmazione
- Non visibile all'utente
- Può avere un forte impatto sui tempi di esecuzione

ESEMPIO: STRUTTURA DATI DIZIONARIO

- Struttura dati generica per memorizzare oggetti
 - Contiene un insieme di **chiavi** univoche
 - Ogni chiave è associata a dei **dati**
 - I dati posso essere duplicati, le chiavi sono uniche
- Operazioni basilari di un Dizionario (prototipo):
 - **SEARCH(Key k)**: cerca e ritorna i dati associati alla chiave k
 - Ritorna NIL se la chiave non è nel dizionario
 - **INSERT(Key k , Data d)**: aggiunge la coppia (k, d) al Dizionario
 - Se (k, d') è nel dizionario, sostituisce solo d' con d
 - **DELETE(Key k)**: elimina la coppia (k, d) dal Dizionario
 - Non fa nulla se k non è nel dizionario

DIZIONARIO: IMPLEMENTAZIONE SU ARRAY

- Idea: usiamo un **array** per memorizzare le coppie (Key,Data)
 - L'ordinamento delle chiavi nell'array è *casuale*
- SEARCH(KEY k)
 - Cerca la chiave k tramite **ricerca lineare** sull'array
 - Ritorna i dati associati a k o NIL
- INSERT(KEY k , DATA d)
 - Verifica con ricerca lineare se k è presente nell'array
 - Se k è nell'array, sostituisce i dati
 - Altrimenti, inserisce la coppia (k, d) nella prima posizione libera
- DELETE(KEY k)
 - Cerca k tramite ricerca lineare e, se k è nell'array
 - Sostituisce la coppia (k, d) con la coppia in fondo all'array

SEARCH SU ARRAY: PSEUDOCODICE

Semplice ricerca lineare su array

```
1: function SEARCH(DICT  $D$ , KEY  $k$ )  $\rightarrow$  DATA
2:    $i = \text{LINSEARCH}(D.A, D.size, k)$ 
3:   if  $i \neq -1$  then
4:     return  $D.A[i].data$ 
5:   else
6:     return NIL
7:
8: function LINSEARCH(<KEY,DATA>  $A[1, \dots, m]$ , INT  $n$ , KEY  $k$ )  $\rightarrow$  INT
9:   for  $i = 1, \dots, n$  do
10:    if  $A[i].key == k$  then
11:      return  $i$ 
12:   return  $-1$   $\triangleright$  key  $k$  not found
```

- $D.A$ = array nel dizionario, $D.size$ = elementi nel dizionario
- Il costo delle operazioni è dato dai costi della ricerca lineare
 - Costo pessimo/medio: $\Theta(n)$
 - Costo ottimo: $O(1)$
- Nota: n è il numero di elementi nell'array non la sua lunghezza

INSERT SU ARRAY: PSEUDOCODICE

■ INSERT su array non ordinato

- 1 Verifica che la chiave non sia già nell'array
- 2 Se la chiave è nell'array, sostituisce i dati
- 3 Altrimenti, inserisce la coppia (k, d) in coda

```
1: function INSERT(DICT  $D$ , KEY  $k$ , DATA  $d$ )  
2:    $i = \text{LINSEARCH}(D.A, D.size, k)$   
3:   if  $i == -1$  then  
4:      $D.size = D.size + 1$                                 ▷ We add one more element  
5:      $i = D.size$                                           ▷ Possible overflow if  $i > m$   
6:      $D.A[i].key = k$   
7:      $D.A[i].data = d$ 
```

- L'inserimento/sostituzione (righe 3-7) ha costo costante
- Il costo delle operazioni è dato dai costi della ricerca lineare
 - Caso pessimo/medio: $\Theta(n)$
 - Caso ottimo: $O(1)$

DELETE SU ARRAY: PSEUDOCODICE

■ DELETE su array non ordinato

- 1 Ricerca lineare della chiave k e, se trovata,
- 2 Sostituisce la coppia (k, d) con la coppia in fondo all'array

```
1: function DELETE(DICT  $D$ , KEY  $k$ )  
2:    $i = \text{LINSEARCH}(D.A, D.size, k)$   
3:   if  $i \neq -1$  then  
4:      $D.A[i].key = D.A[D.size].key$   
5:      $D.A[i].data = D.A[D.size].data$   
6:      $D.size = D.size - 1$ 
```

- Se eseguita, la sostituzione (righe 3-6) ha costo costante
- Il costo delle operazioni è dato dai costi della ricerca lineare
 - Caso pessimo/medio: $\Theta(n)$
 - Caso ottimo: $O(1)$

COSTO DELLE OPERAZIONI SU ARRAY NON ORDINATO

Costi dominati dalla **ricerca lineare**, sempre necessaria

■ **SEARCH**(Key k)

- Ricerca lineare su array non ordinato
- Costo: $O(n)$

■ **INSERT**(Key k , Data d)

- Ricerca lineare su array non ordinato + inserimento
- Costo: $O(n) + O(1) = O(n)$

■ **DELETE**(Key k)

- Ricerca lineare su array non ordinato + sostituzione
- Costo: $O(n) + O(1) = O(n)$

Nota. Non indichiamo se si tratta del caso ottimo/medio/pessimo ma soltanto il limite superiore al costo dell'operazione

DIZIONARIO: IMPLEMENTAZIONE SU ARRAY ORDINATO

- Idea: usiamo un **array** per salvare le coppie (Key,Data) e lo manteniamo **ordinato** rispetto a Key dopo inserimento e rimozione
- SEARCH(KEY k)
 - Cerca la chiave k con **ricerca binaria** sull'array ordinato
 - Ritorna i dati associati a k o NIL
- INSERT(KEY k , DATA d)
 - Cerca con ricerca binaria (modificata) la posizione della chiave k
 - Se k è nell'array, sostituisce i dati, altrimenti
 - sposta di un passo a destra tutte le coppie con chiave $> k$
 - inserisce la coppia (k, d) nello *spazio* aperto con lo shift
- DELETE(KEY k)
 - Cerca la chiave k con ricerca binaria e, se questa è presente
 - Sposta di un passo a sinistra tutte le coppie con chiave $> k$

SEARCH SU ARRAY ORDINATO: PSEUDOCODICE

Semplice ricerca binaria su array ordinato

```
1: function SEARCH(DICT  $D$ , KEY  $k$ )  $\rightarrow$  INT
2:    $i = \text{BINSEARCH}(D.A, D.size, k)$ 
3:   if  $i \neq -1$  then
4:     return  $D.A[i].data$ 
5:   else
6:     return NIL
7:
8: function BINSEARCH(<KEY,DATA>  $A[1, \dots, m]$ , INT  $n$ , KEY  $k$ )  $\rightarrow$  INT
9:    $i = 1, j = n$ 
10:  while  $i \leq j$  do
11:     $M = (i + j)/2$ 
12:    if  $A[M].key == k$  then return  $M$ 
13:    else if  $A[M].key < k$  then  $i = M + 1$ 
14:    else  $j = M - 1$ 
15:  return  $-1$ 
```

- Il costo delle operazioni dipende dai costi della ricerca binaria
 - Costo ottimo: $O(1)$
 - Costo pessimo: $\Theta(\log n)$, Costo medio: $O(\log n)$

INSERT SU ARRAY ORDINATO: PSEUDOCODICE

■ Inserimento su array ordinato

- 1 Ricerca binaria della posizione in cui inserire la chiave k
- 2 Se k non è presente, sposta a destra le coppie con chiave $> k$
- 3 Inserisce la coppia (k, d) nello *spazio* aperto dallo shift

```
1: function INSERT(DICT  $D$ , KEY  $k$ , DATA  $d$ )
2:    $i = \text{BINSEARCHPOS}(D.A, D.size, k)$  ▷  $1 \leq i \leq m$ 
3:   if  $D.A[i].key \neq k$  then
4:     RIGHTSHIFT( $D.A, D.size, i$ )
5:      $D.size = D.size + 1$ 
6:      $D.A[i].key = k$ 
7:      $D.A[i].data = d$ 
8:
9: function RIGHTSHIFT(<KEY,DATA>  $A[1, \dots, m]$ , INT  $n$ , INT  $i$ )
10:  for  $j = n, \dots, i$  do
11:     $A[j + 1] = A[j]$  ▷ Overflow if  $n = m$ 
```

BINSEARCHPOS: PSEUDOCODE

- Come cercare la posizione in cui inserire k in un array ordinato?
 - Se la chiave è già presente, ritorna la posizione della chiave
 - Altrimenti, ritorna la posizione in cui dovrebbe essere inserita

```
1: function BINSEARCHPOS(<KEY,DATA> A[1, ..., m], INT n, KEY k) → INT
2:    $i = 1, j = n$ 
3:   while  $i \leq j$  do
4:      $M = (i + j)/2$ 
5:     if  $A[M].key == k$  then return  $M$ 
6:     else if  $A[M].key < k$  then  $i = M + 1$ 
7:     else  $j = M - 1$ 
8:   return  $i$ 
```

- L'unica differenza con BINSEARCH è a riga 8
- Se k non è in A , il ciclo while termina quando $j < i \Rightarrow j = i - 1$
 - L'unica possibilità è che al passo precedente $i = j$
- Se $i = j$ allora $M = i = j$
 - Se $A[M].key < k$ allora $i = M + 1$ è la posizione di inserimento
 - Se $A[M].key > k$ allora $i = M$ è la posizione di inserimento

INSERT SU ARRAY ORDINATO: ANALISI

- Costo pessimo: $\Theta(n)$
 - Caso pessimo di BINSEARCHPOS: $\Theta(\log n)$
 - Caso pessimo di RIGHTSHIFT: $\Theta(n)$ (primo elemento rimosso)
- Costo ottimo: $O(\log n)$
 - Nel caso ottimo, la posizione di inserimento è in fondo a destra
 - Se BINSEARCHPOS esegue i passi, la posizione più a destra è $n/2^i$
 - Il costo totale è quindi
$$O(i) \text{ (BINSEARCHPOS)} + O(n/2^i) \text{ (RIGHTSHIFT)} = O(i + n/2^i)$$
con $i \in [1, \log_2 n]$
 - Se la chiave k è in fondo all'array $\Rightarrow i = \log_2 n \Rightarrow O(\log n)$
- Costo medio: $\Theta(n)$
 - Assumiamo che ogni posizione di inserimento sia equiprobabile
 - Costo dominato dallo shift: in media spostiamo $n/2$ elementi

DELETE SU ARRAY ORDINATO: PSEUDOCODICE

■ Delete su array ordinato

- 1 Ricerca binaria della chiave k e, se trovata,
- 2 Shift a sinistra di una posizione di tutte le coppie con chiave $> k$

```
1: function DELETE(DICT  $D$ , KEY  $k$ )  
2:    $i = \text{BINSEARCH}(D.A, D.size, k)$   
3:   if  $i \neq -1$  then  
4:     LEFTSHIFT( $D.A, D.size, i$ )  
5:      $D.size = D.size - 1$ 
```

■ Costo pessimo: $\Theta(n)$

- Caso pessimo di BINSEARCH: $\Theta(\log n)$
- Caso pessimo di LEFTSHIFT: $\Theta(n)$ (primo elemento rimosso)

■ Costo ottimo:

- Come caso ottimo di INSERT

■ Costo medio: $O(n)$

- Se la chiave esiste, costo medio di LEFTSHIFT $\Theta(n)$
- Se la chiave non esiste, costo medio di BINSEARCH $O(\log n)$

COSTO DELLE OPERAZIONI SU ARRAY ORDINATO

Costi dominati dallo **shift** per INSERT e DELETE

- **SEARCH**(Key k)

- Ricerca binaria su array ordinato
- Costo: $O(\log n)$

- **INSERT**(Key k , Data d)

- Ricerca binaria su array ordinato + shift
- Costo: $O(\log n) + O(n) = O(n)$

- **DELETE**(Key k)

- Ricerca binaria su array ordinato + shift
- Costo: $O(\log n) + O(n) = O(n)$

Nota. Non indichiamo se si tratta del caso ottimo/medio/pessimo ma soltanto il limite superiore al costo dell'operazione

CONCLUSIONI: ARRAY ORDINATI VS NON ORDINATI

- Siamo partiti con un **prototipo** per la struttura dati Dizionario
- Due strategie **implementative** differenti portano a prestazioni differenti

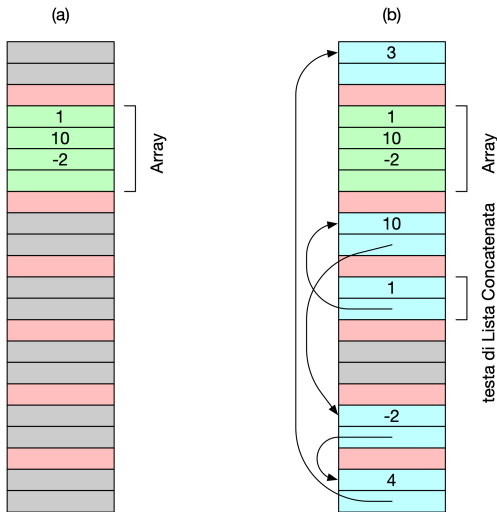
	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$

- Otteniamo qualche miglioramento con array ordinati
 - Miglioramento dovuto allo speed-up della procedura di ricerca
- Abbiamo comunque un dizionario a capienza limitata
- Dobbiamo inventare altre strategie per implementare dizionari che abbiano una capienza illimitata

STRUTTURE DATI ELEMENTARI: LISTA

- Una **Lista** è una struttura dati in cui tutti gli elementi sono organizzati in **ordine sequenziale** (primo elemento, secondo, ...)
- Una lista supporta almeno tre operazioni basilari:
 - Ricerca, Inserimento, Rimozione
- Implementazione con Array
 - L'ordine sequenziale è determinato dagli indici dell'array
 - Lo spazio per gli elementi è allocato staticamente
 - Spazio limitato ma accesso veloce agli elementi
- Implementazione con **Liste concatenate** (Linked Lists)
 - L'ordinamento è determinato da una **catena di puntatori**
 - Lo spazio per gli elementi è allocato dinamicamente su richiesta
 - Costo di accesso dipende dalla posizione ma dimensione illimitata
 - Attribuite a Newell, Shaw, Simon, sviluppatori di IPL (1955-1956)
- Ci concentriamo su diversi tipi di Liste concatenate

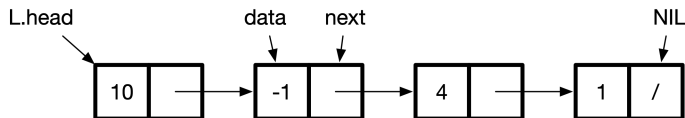
ARRAY VS LISTA CONCATENATA



- (a) Non c'è abbastanza spazio libero contiguo per un array di 5 elementi
- (b) Una lista concatenata non necessita di spazio contiguo

LISTA CONCATENATA SEMPLICE

- Ogni **nodo** x di una **Lista concatenata semplice** contiene
 - $x.data$: dati da memorizzare nella lista
 - $x.next$: un puntatore al nodo successivo nella lista
- Se $x.next = NIL$ allora x è l'ultimo nodo nella lista



- Può essere visitata in un'unica direzione (dalla testa verso la coda)
- In inglese nota come Singly Linked List

COSTO OPERAZIONI LISTA CONCATENATA SEMPLICE

■ SEARCH

- Costo pessimo: $\Theta(n)$ (costo pessimo di una ricerca lineare)
- Costo medio: $\Theta(n)$ (costo medio di una ricerca lineare)
- Costo ottimo: $O(1)$ (costo ottimo di una ricerca lineare)

■ INSERT (inserimento in testa)

- Costo ottimo, medio e pessimo: $O(1)$ (richiede solo l'aggiornamento di pochi puntatori)

■ APPEND (inserimento in coda)

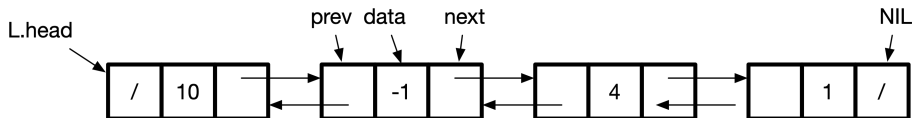
- Costo ottimo, medio e pessimo: $\Theta(n)$ (necessario scorrere tutta la lista per individuare l'ultimo nodo)

■ DELETE

- Caso medio e pessimo: $\Theta(n)$ (come per ricerca lineare: necessario cercare prima il nodo da rimuovere)
- Caso ottimo: $O(1)$ (nodo da rimuovere in testa)

LISTA DOPPIAMENTE CONCATENATA

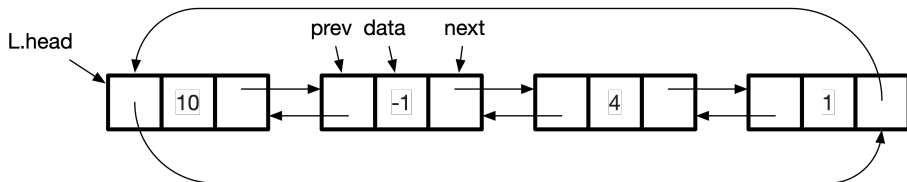
- Sono Liste concatenate semplici in cui ogni nodo contiene anche
 - $x.prev$: un puntatore al nodo precedente nella lista
- Se $x.prev = NIL$ allora x è il primo nodo nella lista



- Può essere visitata in entrambe le direzioni
- Stesso costo delle liste concatenate semplici per tutte le operazioni
- Non necessita ricerca del nodo precedente per la rimozione
 - In ogni nodo abbiamo visibilità sia in avanti che indietro
- In inglese nota come Doubly Linked List

LISTA CONCATENATA CIRCOLARE

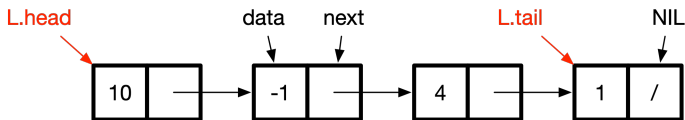
- Solo Liste doppiamente concatenate in cui
 - Il campo *next* dell'ultimo nodo punta al primo nodo
 - Il campo *prev* del primo nodo punta all'ultimo nodo



- Può essere visitata in entrambe le direzioni
- L'accesso alla testa dalla coda è veloce (così come il contrario)
 - Diventa più complesso visitare la lista (nessun puntatore a NIL)
- Stessi costi delle precedenti tranne che per l'inserimento in coda: $O(1)$
- In inglese nota come Circular Linked List

LISTA CON PUNTATORI A TESTA E CODA

- Lista concatenata semplice o doppiamente concatenata in cui
 - Manteniamo puntatori per il nodo in testa e in coda



- L'accesso alla testa e alla coda è veloce
 - Più semplice da gestire rispetto alle liste circolari
- Stessi costi delle liste concatenate circolari
- In inglese nota come Double-Ended List

PSEUDOCODICE DI SEARCH

```
1: function SEARCH(List  $L$ , DATA  $d$ )  $\rightarrow$  BOOL
2:   return SEARCH_NODE( $L$ ,  $d$ )  $\neq$  NIL
3:
4: function SEARCH_NODE(List  $L$ , DATA  $d$ )  $\rightarrow$  NODE
5:    $tmp = L.head$ 
6:   while  $tmp \neq$  NIL do
7:     if  $tmp.data == d$  then
8:       return  $tmp$ 
9:      $tmp = tmp.next$ 
10:  return NIL
```

- Definiamo due funzioni: una che ritorna un riferimento al nodo contenente data (utile dopo) ed uno che verifica unicamente se i dati sono o presenti o meno nella lista
- Costo: stessi costi di una ricerca lineare

PSEUDOCODICE DI INSERT E APPEND

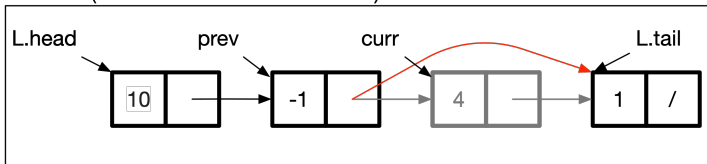
```
1: function INSERT(LIST L, DATA d)
2:   tmp = NODE(d)
3:   if L.head == NIL then
4:     L.head = L.tail = tmp
5:   else
6:     tmp.next = L.head
7:     L.head = tmp
```

```
1: function APPEND(LIST L, DATA d)
2:   tmp = NODE(d)
3:   if L.head == NIL then
4:     L.head = L.tail = tmp
5:   else
6:     L.tail.next = tmp
7:     L.tail = tmp
```

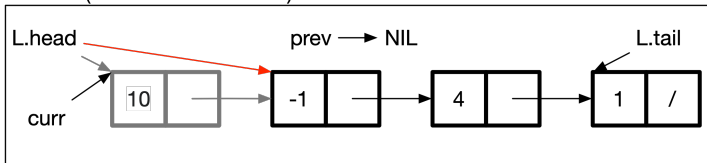
■ Costo: costante in entrambi i casi

RAPPRESENTAZIONE GRAFICA DI DELETE

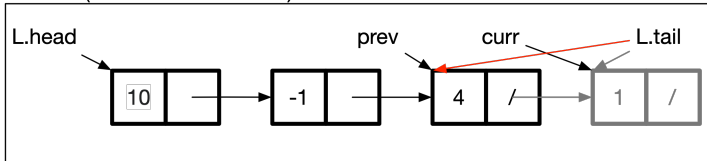
Caso 1 (rimozione di nodo intermedio)



Caso 2 (rimozione della testa)



Caso 3 (rimozione della coda)



PSEUDOCODICE DI DELETE

```
1: function DELETE(LIST L, DATA d)
2:   prev = NIL, curr = L.head
3:   ▷ Find the node to be removed along with its preceding node
4:   while curr ≠ NIL and d ≠ curr.data do
5:     prev = curr
6:     curr = curr.next
7:   ▷ curr = node to be removed, prec = preceding node
8:   if curr ≠ NIL then
9:     if curr == L.head then
10:      L.head = curr.next
11:    else
12:      prev.next = curr.next
13:    if curr == L.tail then
14:      L.tail = prev
```

- Costo: stessi costi di una ricerca lineare (linee 4-6)
- Nota: assumiamo che il nostro linguaggio abbia un garbage collector, quindi non è necessario deallocare esplicitamente il nodo rimosso

LISTE CONCATENATE: RIASSUNTO DEI COSTI

Type	SEARCH	INSERT	APPEND	DELETE
Concatenata semplice	$O(n)$	$O(1)$	$\Theta(n)$	$O(n)$
Doppiamente concatenata	$O(n)$	$O(1)$	$\Theta(n)$	$O(n)$
Circolare	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Puntatori testa e coda	$O(n)$	$O(1)$	$O(1)$	$O(n)$

Limite superiore al costo di ogni operazione

DIZIONARIO CON LISTE CONCATENATE

- Come implementiamo un dizionario su liste concatenate?
- Utilizziamo la precedente implementazione, con le seguenti modifiche
 - La lista memorizza sia un valore chiave k che dei dati d
 - `LLSEARCH(LIST  $L$ , KEY  $k$ )`: è una implementazione della precedente `SEARCH_NODE` con l'unica differenza che cerca solo per valore chiave
 - `LLINSERT(LIST  $L$ , KEY  $k$ , DATA  $d$ )`: è una implementazione della precedente `INSERT` con l'unica differenza che il nodo adesso contiene anche una chiave oltre ai dati
 - `LLDELETE(LIST  $L$ , KEY  $k$ )`: è una implementazione della precedente `DELETE` con l'unica differenza che il nodo adesso viene cercato rispetto al valore chiave

DIZIONARIO CON LISTE CONCATENATE

```
1: function SEARCH(DICT D, KEY k)
2:   tmp = LLSEARCH(D.LL, k)           ▷ Search on Linked List
3:   if tmp == NIL then
4:     return NIL
5:   else
6:     return tmp.data
7:
8: function INSERT(DICT D, KEY k, DATA d)
9:   tmp = LLSEARCH(D.LL, k)           ▷ Search on Linked List
10:  if tmp ≠ NIL then
11:    tmp.data = d
12:  else
13:    LLINSERT(D.LL, k, d)             ▷ Head insert on Linked List
14:
15: function DELETE(DICT D, KEY k)
16:  LLDELETE(D.LL, k)                 ▷ Delete on Linked List
```

- *D.LL* è una struttura dati di tipo Lista concatenata

DIZIONARIO: RIASSUNTO DEI COSTI

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

- Nessun miglioramento rispetto agli approcci precedenti
- Come per gli array non ordinati, la ricerca domina i costi
 - Su array ordinati l'operazione costosa è lo shift
 - Avrebbe senso mantenere la lista concatenata ordinata?
- In ogni caso, con le liste concatenate abbiamo una capienza illimitata

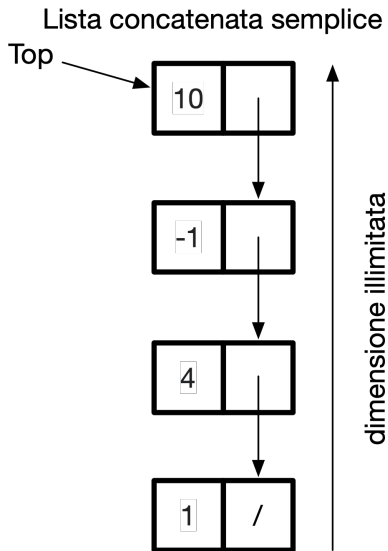
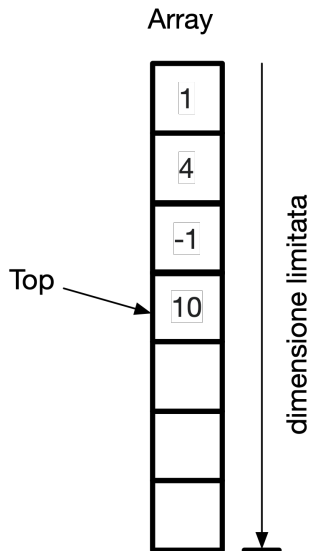
STRUTTURE DATI ELEMENTARI: PILA (STACK)

- Una **Pila** è una struttura dati che supporta due operazioni principali
 - PUSH: aggiunge un nuovo elemento alla struttura
 - POP: rimuove l'elemento aggiunto più di recente
- Intuitivamente, una pila di elementi uno un cima all'altro
 - Ad esempio, una pila di piatti
 - Modalità LIFO (Last In First Out)
- Applicazioni delle pile
 - Gestione di record di attivazione (chiamate a funzione)
 - Linguaggi stack-oriented (PostScript, BibTex, ...)
 - Numerose applicazioni in algoritmi
 - Editor di testo (operazioni undo e redo)
 - Syntax parsing (parentesi bilanciate)
 - ...

IMPLEMENTAZIONE DELLA STRUTTURA DATI PILA

- Una Pila è una Lista che supporta un numero limitato di operazioni
- Implementazione con Liste concatenate semplici (esercizio)
 - POP: rimuove la testa della lista
 - PUSH: inserisce l'elemento in testa alla lista
 - Pro: dimensione illimitata
 - Con: piccolo overhead di memoria (valore+puntatore)
 - Domanda: perchè non usare Liste doppiamente concatenate?
- Implementazione con Array
 - POP: rimuove l'ultimo elemento nell'array
 - PUSH: inserisce l'elemento nella prima posizione libera
 - Pro: nessun overhead di memoria (memorizza solo il valore)
 - Con: dimensione limitata
- In entrambi i casi POP and PUSH costano $O(1)$

IMPLEMENTAZIONE DELLA STRUTTURA DATI PILA

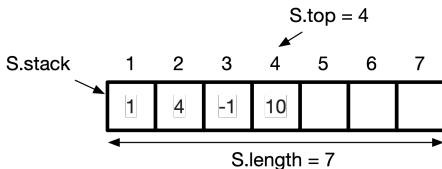


PUSH AND POP CON ARRAY STATICO

```
1: function PUSH(STACK S, INT x)
2:   if S.top == S.length then
3:     error "overflow"
4:   else
5:     S.top = S.top + 1
6:     S.stack[S.top] = x
```

```
1: function POP(STACK S) → INT
2:   if S.top == 0 then
3:     error "underflow"
4:   else
5:     e = S.stack[S.top]
6:     S.top = S.top - 1
7:   return e
```

- Entrambe le funzioni costano: ?
- Stack underflow causato da un uso poco attento di POP
- Stack overflow causato da mancanza di spazio
- Come implementare una Pila con array dinamico?

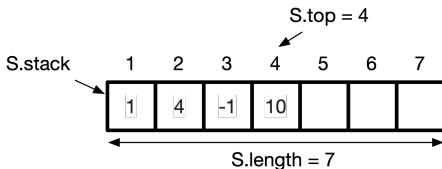


PUSH AND POP CON ARRAY STATICO

```
1: function PUSH(STACK S, INT x)
2:   if S.top == S.length then
3:     error "overflow"
4:   else
5:     S.top = S.top + 1
6:     S.stack[S.top] = x
```

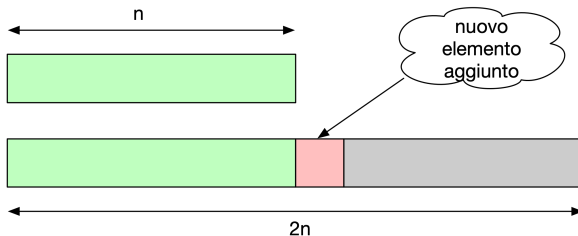
```
1: function POP(STACK S) → INT
2:   if S.top == 0 then
3:     error "underflow"
4:   else
5:     e = S.stack[S.top]
6:     S.top = S.top - 1
7:   return e
```

- Entrambe le funzioni costano (pessimo e ottimo) $O(1)$
- Stack underflow causato da un uso poco attento di POP
- Stack overflow causato da mancanza di spazio
- Come implementare una Pila con array dinamico?

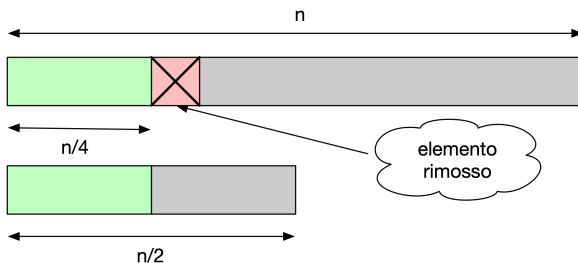


STRATEGIA CON ARRAY DINAMICO

- Raddoppiamo la dimensione dell'array quando non c'è spazio libero



- Dimezziamo la dimensione dell'array quando l'occupazione è di $1/4$



POP CON ARRAY DINAMICO

```
1: function POP(STACK S) → INT
2:   if S.top == 0 then
3:     error "underflow"
4:   else
5:     e = S.stack[S.top]
6:     S.top = S.top - 1
7:     if S.top ≤ ⌊S.length/4⌋ then
8:       n = S.length
9:       LET T[1, ..., ⌈n/2⌋] BE A NEW ARRAY
10:      for i = 1, ..., ⌊n/4⌋ do
11:        T[i] = S.stack[i]
12:      S.stack = T
13:      S.length = ⌈n/2⌋
14:    return e
```

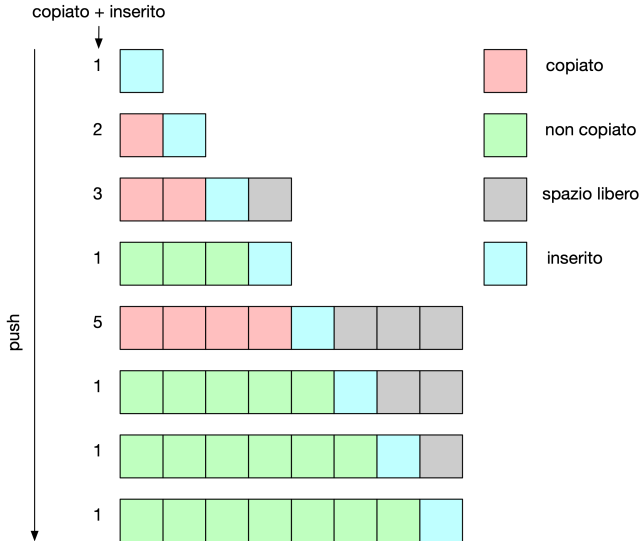
- Costo nel caso pessimo: $\Theta(n)$ (copia dell'array, linee 10 – 11)
- Costo nel caso ottimo: $O(1)$ (array utilizzato per più di 1/4)

PUSH CON ARRAY DINAMICO

```
1: function PUSH(STACK S, INT x)
2:   if S.top == S.length then
3:     n = S.length
4:     LET T[1, ..., 2n] BE A NEW ARRAY
5:     for i = 1, ..., n do
6:       T[i] = S.stack[i]
7:     S.stack = T
8:     S.length = 2n
9:   S.top = S.top + 1
10:  S.stack[S.top] = x
```

- Costo nel caso pessimo: $\Theta(n)$ (copia dell'array, linee 5 – 6)
- Costo nel caso ottimo: $O(1)$ (array non pieno)

ANALISI DI PUSH



ANALISI AMMORTIZZATA DI PUSH

- Costo nel caso pessimo $\Theta(n)$, costo nel caso ottimo $O(1)$
- Qual è il costo di n PUSH partendo da una Pila vuota?
 - Il costo nel caso pessimo è $O(n^2)$
 - $O(n^2)$ non è una stima accurata: non raddoppiamo spesso l'array
 - Il costo del' i -esima PUSH è infatti

$$c_i = \begin{cases} i & \text{se } i-1 \text{ è una potenza esatta di 2} \\ 1 & \text{altrimenti} \end{cases}$$

- Metodo degli aggregati: il costo totale di n PUSH è

$$\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\log_2 n} 2^j = n + \frac{2^{\log_2 n + 1} - 1}{2 - 1} = 3n - 1 = O(n)$$

- Il **costo ammortizzato** di n PUSH è dunque $\frac{O(n)}{n} = O(1)$

ANALISI AMMORTIZZATA DI PUSH E POP

- Così come abbiamo fatto per PUSH possiamo dimostrare che il costo ammortizzato di n POP partendo da una Pila piena è $O(1)$
 - Non abbiamo una caratterizzazione semplice come per PUSH
 - Dobbiamo utilizzare il metodo degli accantonamenti
 - Costo ammortizzato: 2ϵ (1ϵ rimozione + 1ϵ credito per copia)
- Quanto costa una generica sequenza di n PUSH e POP?
 - Di nuovo, metodo degli accantonamenti (non aggregati)
 - Possiamo dimostrare che il costo di **ogni sequenza di n PUSH e POP** su array dinamico inizialmente vuoto è al peggio $O(n)$
 $\Rightarrow$ **costo ammortizzato di $O(1)$** per entrambe PUSH e POP
- **N.B.** Tali costi ammortizzati valgono solo per la nostra strategia di espansione/contrazione ma non per ogni possibile strategia
 - Ad esempio, dimezzare l'array quando è pieno per metà porta a costi ammortizzati di $O(n)$ per operazione

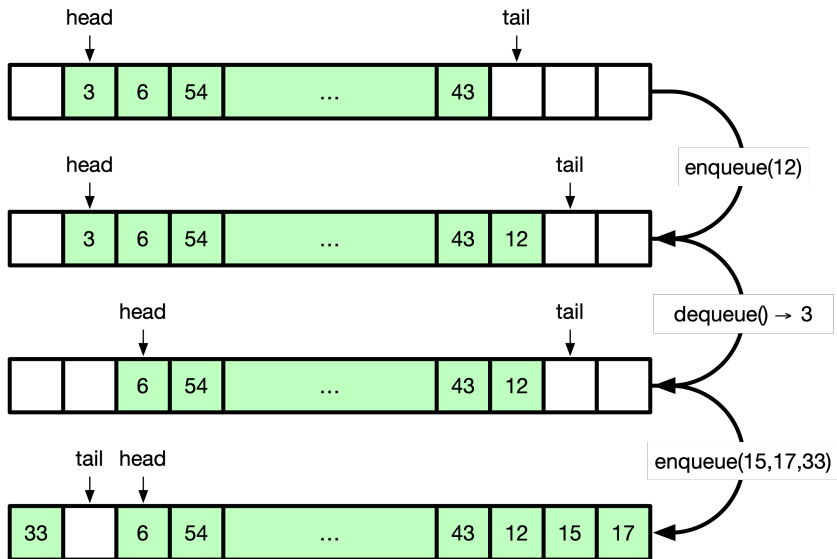
STRUTTURE DATI ELEMENTARI: CODA (QUEUE)

- Una **Coda** è una struttura dati che supporta due operazioni principali
 - ENQUEUE: aggiunge un elemento in fondo alla coda
 - DEQUEUE: rimuove l'elemento in testa alla coda
- Gli elementi sono rimossi nello stesso ordine in cui sono inseriti
 - Non è possibile accedere ad elementi nel mezzo della Coda
- Intuitivamente, una coda rappresenta una fila di elementi
 - Ad esempio, una fila di persone in attesa di un servizio
 - Modalità FIFO (First In First Out)
- Applicazioni della coda
 - Scheduling dei processi nei sistemi operativi
 - Visita di tipo Bread-first-search su grafi
 - ...

IMPLEMENTAZIONE DELLA STRUTTURA DATI CODA

- Una Coda è una Lista che supporta un numero limitato di operazioni
- Implementazione con Liste concatenate circolari (esercizio)
 - ENQUEUE: inserisce l'elemento in coda alla lista
 - DEQUEUE: rimuove la testa della lista
 - Pro: dimensione illimitata
 - Con: overhead di memoria (valore+2 puntatori)
- Implementazione con Liste concatenate semplici (esercizio)
 - Usiamo la versione con puntatore a testa e coda
 - ENQUEUE: inserisce in coda usando il puntatore alla coda
 - Pro e Con come per Liste concatenate circolari
- Implementazione con **Array circolari**
 - Pro: nessun overhead di memoria (memorizza solo il valore)
 - Con: dimensione limitata
- In tutti e tre i casi ENQUEUE and DEQUEUE costano $O(1)$

IMPLEMENTAZIONE CON ARRAY CIRCOLARE



IMPLEMENTAZIONE CON ARRAY CIRCOLARE

```
1: function ENQUEUE(Queue Q, INT x)
2:   if Q.size == Q.length then
3:     error "overflow"
4:   Q.buf[Q.tail] = x
5:   Q.tail = (Q.tail % Q.length) + 1
6:   Q.size = Q.size + 1
```

```
1: function DEQUEUE(Queue Q) → INT
2:   if Q.size == 0 then      ▷ Empty Q
3:     error "underflow"
4:   x = Q.buf[Q.head]
5:   Q.head = (Q.head % Q.length) + 1
6:   Q.size = Q.size - 1
7:   return x
```

- Entrambe costano (ot-timo, medio, pessimo) $O(1)$
- $Q.size$ è il numero di elementi in Q
- $tail$ punta alla prima cella libera
- $\%$ = operazione modulo
- Il modulo permette una visita circolare dell'array
- Come implementare una struttura dati Coda con array dinamico circolare?